Додаток 2. Результати тестування

Valid Basic

examples\valid-basic.signal

```
1 PROGRAM HELLO;
2 CONST
3   X = '42';
4 BEGIN
5 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	HELLO
59	1	14	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	42
39	3	10	Delimiter	'
59	3	11	Delimiter	;
302	4	1	Keyword	BEGIN
303	5	1	Keyword	END
46	5	4	Delimiter	.

Identifiers

Code	Value
1000	HELLO
1001	X

Literals

Code	Value
500	42

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─HELLO
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     ├─(13)─<identifier>
                  │     │           │     │      └─(15)─X
                  │     │           │     └─(9)─'<complex-number>'
                  │     │           │           └─(10)─<left-part> <right-part>
                  │     │           │                 ├─(11)─<unsigned-integer>
                  │     │           │                 │      └─(17)─42
                  │     │           │                 └─(12)─<empty>
                  │     │           └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Valid Basic With Tabs

examples\valid-basic-with-tabs.signal

```
1 PROGRAM TABS;
2 CONST
3         X       =       '1';
4         Y =     '2$EXP(3)';
5                 Z       = '4,5';
6 BEGIN
7 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	TABS
59	1	13	Delimiter	;
301	2	1	Keyword	CONST
1001	3	5	Identifier	X
61	3	9	Delimiter	=
39	3	13	Delimiter	'
500	3	14	Literal	1
39	3	15	Delimiter	'
59	3	16	Delimiter	;
1002	4	5	Identifier	Y
61	4	7	Delimiter	=
39	4	9	Delimiter	'
501	4	10	Literal	2
256	4	11	Multi-Delimiter	$EXP
40	4	15	Delimiter	(
502	4	16	Literal	3
41	4	17	Delimiter	)
39	4	18	Delimiter	'
59	4	19	Delimiter	;
1003	5	9	Identifier	Z
61	5	13	Delimiter	=
39	5	15	Delimiter	'
503	5	16	Literal	4
44	5	17	Delimiter	,
504	5	18	Literal	5
39	5	19	Delimiter	'
59	5	20	Delimiter	;
302	6	1	Keyword	BEGIN
303	7	1	Keyword	END
46	7	4	Delimiter	.

Identifiers

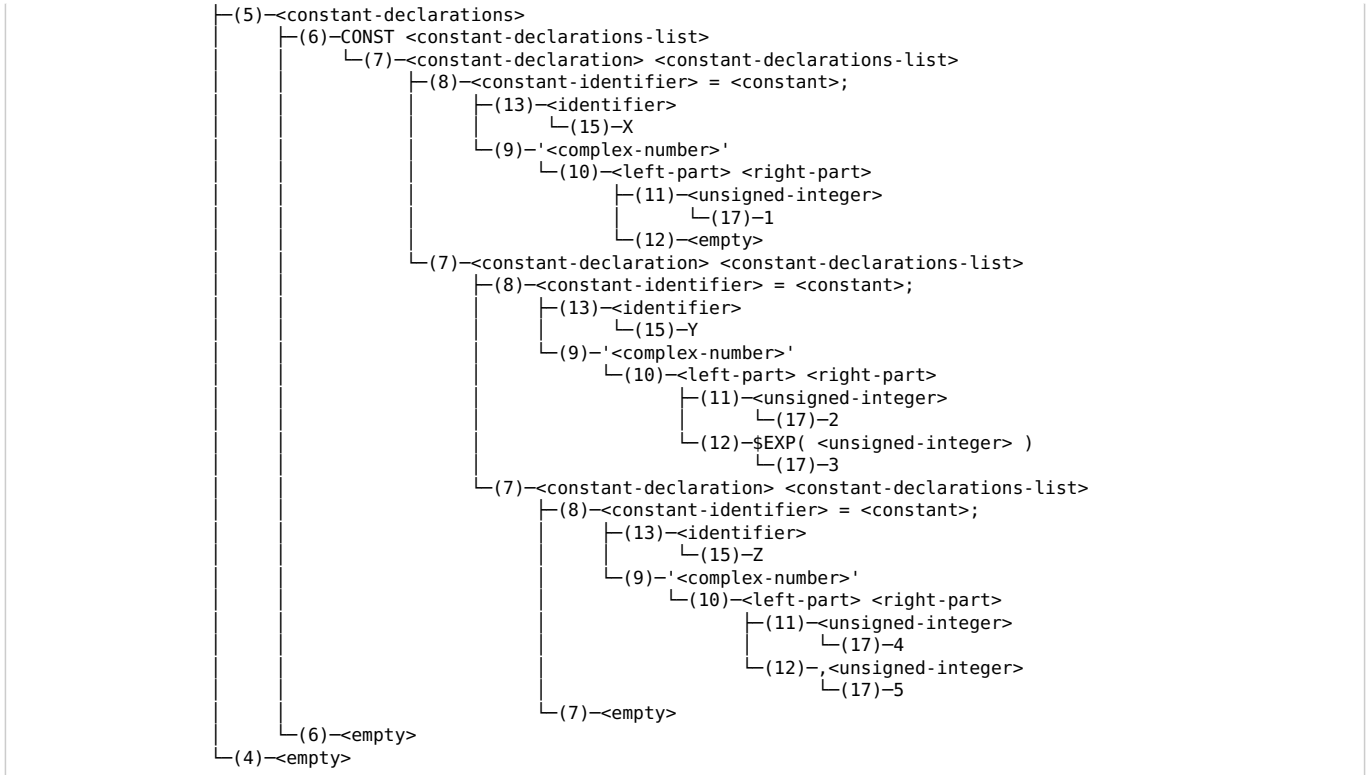
Code	Value
1000	TABS
1001	X
1002	Y
1003	Z

Literals

Code	Value
500	1
501	2
502	3
503	4
504	5

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─TABS
            └─(3)─<declarations> BEGIN <statements-list> END
```


(5)—<constant-declarations>
(6)—CONST <constant-declarations-list>
(7)—<constant-declaration> <constant-declarations-list>
(8)—<constant-identifier> = <constant>;
(13)—<identifier>
(15)—X
(9)—'<complex-number>'
(10)—<left-part> <right-part>
(11)—<unsigned-integer>
(17)—1
(12)—<empty>
(7)—<constant-declaration> <constant-declarations-list>
(8)—<constant-identifier> = <constant>;
(13)—<identifier>
(15)—Y
(9)—'<complex-number>'
(10)—<left-part> <right-part>
(11)—<unsigned-integer>
(17)—2
(12)—$EXP(<unsigned-integer>)
(17)—3
(7)—<constant-declaration> <constant-declarations-list>
(8)—<constant-identifier> = <constant>;
(13)—<identifier>
(15)—Z
(9)—'<complex-number>'
(10)—<left-part> <right-part>
(11)—<unsigned-integer>
(17)—4
(12)—,<unsigned-integer>
(17)—5
(7)—<empty>
(6)—<empty>
(4)—<empty>

Valid Comments

examples\valid-comments.signal

```
(* A lot of comments *)
PROGRAM COMMENTS;
CONST
  (* Single-line comment *)
  (*
    Multi-line comment
  *)
  (**) (*  *)
  (* Multiple *) (* comments *) (* on *) (* the *) (* same *) (* line *)
  (*****) (*(()*())*) (*;.:*) (* Comments with nested * and () *)
  (* Invalid symbols are allowed! %#^@& *)
BEGIN (* on the same like as identifier *)
END (* between tokens *) .
```

Output

```
Tokens
┌──────┬─────┬─────┬────────────┬──────────────────────────────────────────┐
│ Code │ Row │ Col │ Type       │ Value                                    │
├──────┼─────┼─────┼────────────┼──────────────────────────────────────────┤
│  300 │   2 │   1 │ Keyword    │ PROGRAM                                  │
│ 1000 │   2 │   9 │ Identifier │ COMMENTS                                 │
│   59 │   2 │  17 │ Delimiter  │ ;                                        │
│  301 │   3 │   1 │ Keyword    │ CONST                                    │
│  302 │  12 │   1 │ Keyword    │ BEGIN                                    │
│  303 │  13 │   1 │ Keyword    │ END                                      │
│   46 │  13 │  26 │ Delimiter  │ .                                        │
└──────┴─────┴─────┴────────────┴──────────────────────────────────────────┘

Identifiers
┌──────┬──────────┐
│ Code │ Value    │
├──────┼──────────┤
│ 1000 │ COMMENTS │
└──────┴──────────┘

Literals
┌──────┬───────┐
│ Code │ Value │
├──────┼───────┤
└──────┴───────┘

Syntax Tree
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─COMMENTS
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Valid Constants

examples\valid-constants.signal

```
(* All ways to declare a constant within a grammar *)
PROGRAM CONSTANTS;
CONST
  (* Single-line comment *)
  A = '';
  A = '1';
  A = '2,3';
  B = ',4';
  B = '5$EXP(6)';
  C = '$EXP(7)';
BEGIN
  (*
    Multi-line comment
  *)
END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	CONSTANTS
59	2	18	Delimiter	;
301	3	1	Keyword	CONST
1001	5	3	Identifier	A
61	5	5	Delimiter	=
39	5	7	Delimiter	'
39	5	8	Delimiter	'
59	5	9	Delimiter	;
1001	6	3	Identifier	A
61	6	5	Delimiter	=
39	6	7	Delimiter	'
500	6	8	Literal	1
39	6	9	Delimiter	'
59	6	10	Delimiter	;
1001	7	3	Identifier	A
61	7	5	Delimiter	=
39	7	7	Delimiter	'
501	7	8	Literal	2
44	7	9	Delimiter	,
502	7	10	Literal	3
39	7	11	Delimiter	'
59	7	12	Delimiter	;
1002	8	3	Identifier	B
61	8	5	Delimiter	=
39	8	7	Delimiter	'
44	8	8	Delimiter	,
503	8	9	Literal	4
39	8	10	Delimiter	'
59	8	11	Delimiter	;
1002	9	3	Identifier	B
61	9	5	Delimiter	=
39	9	7	Delimiter	'
504	9	8	Literal	5
256	9	9	Multi-Delimiter	$EXP
40	9	13	Delimiter	(
505	9	14	Literal	6
41	9	15	Delimiter	)
39	9	16	Delimiter	'
59	9	17	Delimiter	;
1003	10	3	Identifier	C
61	10	5	Delimiter	=
39	10	7	Delimiter	'
256	10	8	Multi-Delimiter	$EXP
40	10	12	Delimiter	(
506	10	13	Literal	7
41	10	14	Delimiter	)
39	10	15	Delimiter	'
59	10	16	Delimiter	;
302	11	1	Keyword	BEGIN
303	15	1	Keyword	END

46	15	4	Delimiter	.

Identifiers

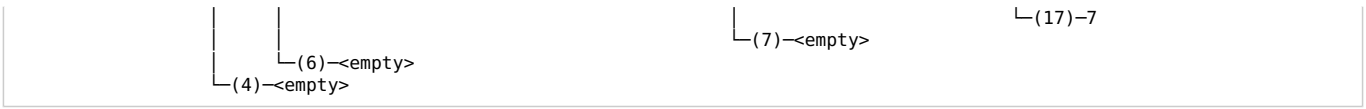
Code	Value
1000	CONSTANTS
1001	A
1002	B
1003	C

Literals

Code	Value
500	1
501	2
502	3
503	4
504	5
505	6
506	7

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─CONSTANTS
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     ├─(13)─<identifier>
                  │     │           │     │      └─(15)─A
                  │     │           │     └─(9)─'<complex-number>'
                  │     │           │           └─(10)─<left-part> <right-part>
                  │     │           │                 ├─(11)─<empty>
                  │     │           │                 └─(12)─<empty>
                  │     │           └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                 ├─(8)─<constant-identifier> = <constant>;
                  │     │                 │     ├─(13)─<identifier>
                  │     │                 │     │      └─(15)─A
                  │     │                 │     └─(9)─'<complex-number>'
                  │     │                 │           └─(10)─<left-part> <right-part>
                  │     │                 │                 ├─(11)─<unsigned-integer>
                  │     │                 │                 │      └─(17)─1
                  │     │                 │                 └─(12)─<empty>
                  │     │                 └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                       ├─(8)─<constant-identifier> = <constant>;
                  │     │                       │     ├─(13)─<identifier>
                  │     │                       │     │      └─(15)─A
                  │     │                       │     └─(9)─'<complex-number>'
                  │     │                       │           └─(10)─<left-part> <right-part>
                  │     │                       │                 ├─(11)─<unsigned-integer>
                  │     │                       │                 │      └─(17)─2
                  │     │                       │                 └─(12)─,<unsigned-integer>
                  │     │                       │                        └─(17)─3
                  │     │                       └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                             ├─(8)─<constant-identifier> = <constant>;
                  │     │                             │     ├─(13)─<identifier>
                  │     │                             │     │      └─(15)─B
                  │     │                             │     └─(9)─'<complex-number>'
                  │     │                             │           └─(10)─<left-part> <right-part>
                  │     │                             │                 ├─(11)─<empty>
                  │     │                             │                 └─(12)─,<unsigned-integer>
                  │     │                             │                        └─(17)─4
                  │     │                             └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                   ├─(8)─<constant-identifier> = <constant>;
                  │     │                                   │     ├─(13)─<identifier>
                  │     │                                   │     │      └─(15)─B
                  │     │                                   │     └─(9)─'<complex-number>'
                  │     │                                   │           └─(10)─<left-part> <right-part>
                  │     │                                   │                 ├─(11)─<unsigned-integer>
                  │     │                                   │                 │      └─(17)─5
                  │     │                                   │                 └─(12)─$EXP( <unsigned-integer> )
                  │     │                                   │                        └─(17)─6
                  │     │                                   └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                         ├─(8)─<constant-identifier> = <constant>;
                  │     │                                         │     ├─(13)─<identifier>
                  │     │                                         │     │      └─(15)─C
                  │     │                                         │     └─(9)─'<complex-number>'
                  │     │                                         │           └─(10)─<left-part> <right-part>
                  │     │                                         │                 ├─(11)─<empty>
                  │     │                                         │                 └─(12)─$EXP( <unsigned-integer> )
```


└(17)—7
└(7)—<empty>
└(6)—<empty>
└(4)—<empty>

Valid Edge Cases

examples\valid-edge-cases.signal

```
(* Edge cases within the grammar *)
PROGRAM EDGES;
CONST
  AB12CD = '999,000';
  Z = '100$EXP(200)';
  LONGNAME123 = '0';
  A = '0,0';
BEGIN
END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	EDGES
59	2	14	Delimiter	;
301	3	1	Keyword	CONST
1001	4	3	Identifier	AB12CD
61	4	10	Delimiter	=
39	4	12	Delimiter	'
500	4	13	Literal	999
44	4	16	Delimiter	,
501	4	17	Literal	000
39	4	20	Delimiter	'
59	4	21	Delimiter	;
1002	5	3	Identifier	Z
61	5	5	Delimiter	=
39	5	7	Delimiter	'
502	5	8	Literal	100
256	5	11	Multi-Delimiter	$EXP
40	5	15	Delimiter	(
503	5	16	Literal	200
41	5	19	Delimiter	)
39	5	20	Delimiter	'
59	5	21	Delimiter	;
1003	6	3	Identifier	LONGNAME123
61	6	15	Delimiter	=
39	6	17	Delimiter	'
504	6	18	Literal	0
39	6	19	Delimiter	'
59	6	20	Delimiter	;
1004	7	3	Identifier	A
61	7	5	Delimiter	=
39	7	7	Delimiter	'
504	7	8	Literal	0
44	7	9	Delimiter	,
504	7	10	Literal	0
39	7	11	Delimiter	'
59	7	12	Delimiter	;
302	8	1	Keyword	BEGIN
303	9	1	Keyword	END
46	9	4	Delimiter	.

Identifiers

Code	Value
1000	EDGES
1001	AB12CD
1002	Z
1003	LONGNAME123
1004	A

Literals

Code	Value
500	999
501	000
502	100

```
|   503 | 200                                |
|   504 | 0                                  |
└───────┴────────────────────────────────────┘

Syntax Tree
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─EDGES
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     ├─(13)─<identifier>
                  │     │           │     │      └─(15)─AB12CD
                  │     │           │     └─(9)─'<complex-number>'
                  │     │           │           └─(10)─<left-part> <right-part>
                  │     │           │                 ├─(11)─<unsigned-integer>
                  │     │           │                 │      └─(17)─999
                  │     │           │                 └─(12)─,<unsigned-integer>
                  │     │           │                        └─(17)─000
                  │     │           └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                 ├─(8)─<constant-identifier> = <constant>;
                  │     │                 │     ├─(13)─<identifier>
                  │     │                 │     │      └─(15)─Z
                  │     │                 │     └─(9)─'<complex-number>'
                  │     │                 │           └─(10)─<left-part> <right-part>
                  │     │                 │                 ├─(11)─<unsigned-integer>
                  │     │                 │                 │      └─(17)─100
                  │     │                 │                 └─(12)─$EXP( <unsigned-integer> )
                  │     │                 │                        └─(17)─200
                  │     │                 └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                       ├─(8)─<constant-identifier> = <constant>;
                  │     │                       │     ├─(13)─<identifier>
                  │     │                       │     │      └─(15)─LONGNAME123
                  │     │                       │     └─(9)─'<complex-number>'
                  │     │                       │           └─(10)─<left-part> <right-part>
                  │     │                       │                 ├─(11)─<unsigned-integer>
                  │     │                       │                 │      └─(17)─0
                  │     │                       │                 └─(12)─<empty>
                  │     │                       └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                             ├─(8)─<constant-identifier> = <constant>;
                  │     │                             │     ├─(13)─<identifier>
                  │     │                             │     │      └─(15)─A
                  │     │                             │     └─(9)─'<complex-number>'
                  │     │                             │           └─(10)─<left-part> <right-part>
                  │     │                             │                 ├─(11)─<unsigned-integer>
                  │     │                             │                 │      └─(17)─0
                  │     │                             │                 └─(12)─,<unsigned-integer>
                  │     │                             │                        └─(17)─0
                  │     │                             └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Valid Empty

examples\valid-empty.signal

```
1 (* Empty program *)
2 PROGRAM EMPTY;
3 BEGIN
4 END.
```

Output

Tokens

Code	Row	Col	Type	Value
300	2	1	Keyword	PROGRAM
1000	2	9	Identifier	EMPTY
59	2	14	Delimiter	;
302	3	1	Keyword	BEGIN
303	4	1	Keyword	END
46	4	4	Delimiter	.

Identifiers

Code	Value
1000	EMPTY

Literals

Code	Value

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─EMPTY
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Syntax Error Extraneous Symbols

examples\syntax-error-extraneous-symbols.signal

```
1 CONST PROGRAM PRG; (* Extraneous CONST keyword in procedure declaration *)
2 CONST
3   X == '42'; (* Extraneous '=' *)
4   Y = ''24'; (* Extraneous opening ''' (single quote) *)
5   Z = '54,$EXP(32)'; (* Extraneous ',' in constant *)
6   R R = '23,45'; (* Duplicate constant identifier *)
7   Q = '999';
8 BEGIN
9 END. PRG (* Extraneous identifier*)
```

Output

```
Parser | Error [1:1]: Program must start with 'PROGRAM' keyword
Parser | Error [1:7]: Expected constant identifier in constant declaration
Parser | Error [2:1]: Expected constant identifier in constant declaration
Parser | Error [4:9]: Expected ';' at the end of constant declaration
Parser | Error [4:9]: Expected constant identifier in constant declaration
Parser | Error [5:11]: Expected unsigned integer literal
Parser | Error [6:5]: Expected '=' in constant declaration
Parser | Error [9:6]: Unexpected symbols after end of program

Tokens
```

Code	Row	Col	Type	Value
301	1	1	Keyword	CONST
300	1	7	Keyword	PROGRAM
1000	1	15	Identifier	PRG
59	1	18	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
61	3	6	Delimiter	=
39	3	8	Delimiter	'
500	3	9	Literal	42
39	3	11	Delimiter	'
59	3	12	Delimiter	;
1002	4	3	Identifier	Y
61	4	5	Delimiter	=
39	4	7	Delimiter	'
39	4	8	Delimiter	'
501	4	9	Literal	24
39	4	11	Delimiter	'
59	4	12	Delimiter	;
1003	5	3	Identifier	Z
61	5	5	Delimiter	=
39	5	7	Delimiter	'
502	5	8	Literal	54
44	5	10	Delimiter	,
256	5	11	Multi-Delimiter	$EXP
40	5	15	Delimiter	(
503	5	16	Literal	32
41	5	18	Delimiter	)
39	5	19	Delimiter	'
59	5	20	Delimiter	;
1004	6	3	Identifier	R
1004	6	5	Identifier	R
61	6	7	Delimiter	=
39	6	9	Delimiter	'
504	6	10	Literal	23
44	6	12	Delimiter	,
505	6	13	Literal	45
39	6	15	Delimiter	'
59	6	16	Delimiter	;
1005	7	3	Identifier	Q
61	7	5	Delimiter	=
39	7	7	Delimiter	'
506	7	8	Literal	999
39	7	11	Delimiter	'
59	7	12	Delimiter	;
302	8	1	Keyword	BEGIN
303	9	1	Keyword	END
46	9	4	Delimiter	.
1000	9	6	Identifier	PRG

Identifiers

Code	Value
1000	PRG
1001	X
1002	Y
1003	Z
1004	R
1005	Q

Literals

Code	Value
500	42
501	24
502	54
503	32
504	23
505	45
506	999

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     └─(13)─<identifier>
                  │     │           └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                 ├─(8)─<constant-identifier> = <constant>;
                  │     │                 │     └─(13)─<identifier>
                  │     │                 └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                       ├─(8)─<constant-identifier> = <constant>;
                  │     │                       │     ├─(13)─<identifier>
                  │     │                       │     │      └─(15)─Y
                  │     │                       │     └─(9)─'<complex-number>'
                  │     │                       │           └─(10)─<left-part> <right-part>
                  │     │                       │                 ├─(11)─<empty>
                  │     │                       │                 └─(12)─<empty>
                  │     │                       └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                             ├─(8)─<constant-identifier> = <constant>;
                  │     │                             │     └─(13)─<identifier>
                  │     │                             └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                   ├─(8)─<constant-identifier> = <constant>;
                  │     │                                   │     ├─(13)─<identifier>
                  │     │                                   │     │      └─(15)─Z
                  │     │                                   │     └─(9)─'<complex-number>'
                  │     │                                   │           └─(10)─<left-part> <right-part>
                  │     │                                   │                 ├─(11)─<unsigned-integer>
                  │     │                                   │                 │      └─(17)─54
                  │     │                                   │                 └─(12)─,<unsigned-integer>
                  │     │                                   └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                         ├─(8)─<constant-identifier> = <constant>;
                  │     │                                         │     └─(13)─<identifier>
                  │     │                                         │            └─(15)─R
                  │     │                                         └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                               ├─(8)─<constant-identifier> = <constant>;
                  │     │                                               │     ├─(13)─<identifier>
                  │     │                                               │     │      └─(15)─Q
                  │     │                                               │     └─(9)─'<complex-number>'
                  │     │                                               │           └─(10)─<left-part> <right-part>
                  │     │                                               │                 ├─(11)─<unsigned-integer>
                  │     │                                               │                 │      └─(17)─999
                  │     │                                               │                 └─(12)─<empty>
                  │     │                                               └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Syntax Error Mangled

examples\syntax-error-mangled.signal

```
1 PROGRA TEST; (* Mangled 'PROGRAM' keyword *)
2 CONST
3   A = '1,2';
4   B = '3' (* Missing ';' after constant *)
5   CONST = '4'; (* CONST keyword used as an identifier *)
6 BEGIN
7 eND. (* Mangled 'END' keyword *)
```

Output

```
Tokenizer | Error [7:1]: Invalid character 'e'
Parser | Error [1:1]: Program must start with 'PROGRAM' keyword
Parser | Error [5:3]: Expected ';' at the end of constant declaration
Parser | Error [5:3]: Expected constant identifier in constant declaration
Parser | Error [7:2]: Expected 'END' keyword at the end of the block

Tokens
```

Code	Row	Col	Type	Value
1000	1	1	Identifier	PROGRA
1001	1	8	Identifier	TEST
59	1	12	Delimiter	;
301	2	1	Keyword	CONST
1002	3	3	Identifier	A
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	1
44	3	9	Delimiter	,
501	3	10	Literal	2
39	3	11	Delimiter	'
59	3	12	Delimiter	;
1003	4	3	Identifier	B
61	4	5	Delimiter	=
39	4	7	Delimiter	'
502	4	8	Literal	3
39	4	9	Delimiter	'
301	5	3	Keyword	CONST
61	5	9	Delimiter	=
39	5	11	Delimiter	'
503	5	12	Literal	4
39	5	13	Delimiter	'
59	5	14	Delimiter	;
302	6	1	Keyword	BEGIN
1004	7	2	Identifier	ND
46	7	4	Delimiter	.

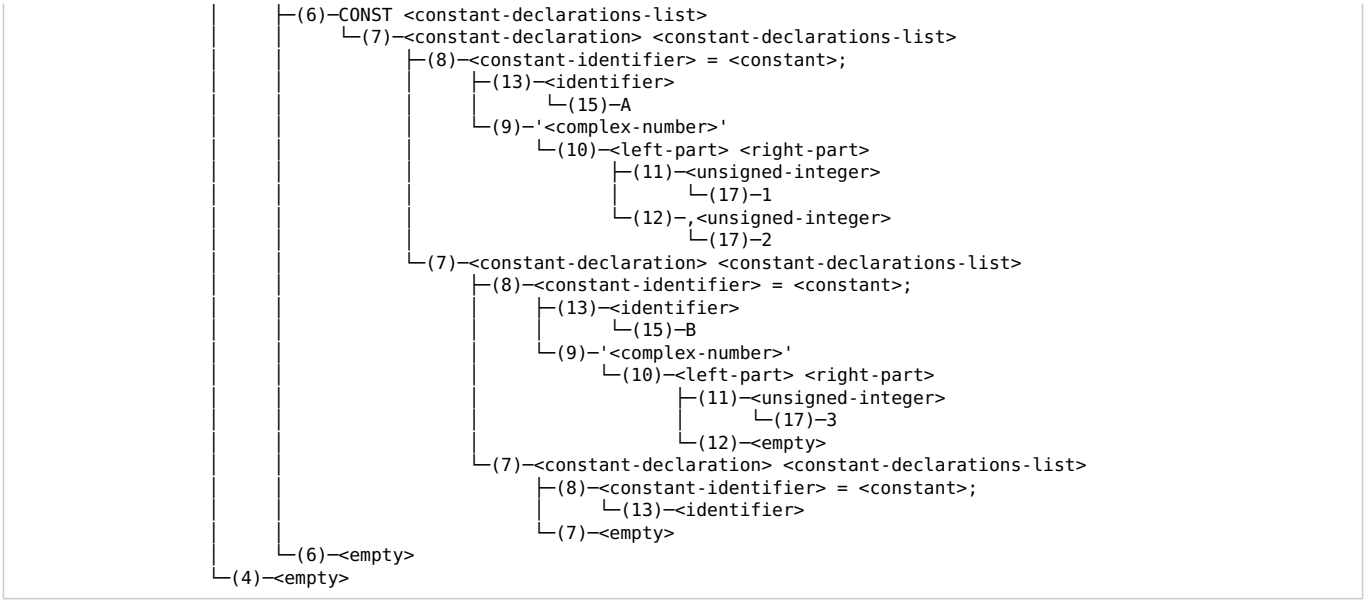
```
Identifiers
```

Code	Value
1000	PROGRA
1001	TEST
1002	A
1003	B
1004	ND

```
Literals
```

Code	Value
500	1
501	2
502	3
503	4

```
Syntax Tree
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
```


├─(6)─CONST <constant-declarations-list>
└─(7)─<constant-declaration> <constant-declarations-list>
├─(8)─<constant-identifier> = <constant>;
├─(13)─<identifier>
└─(15)─A
└─(9)─'<complex-number>'
└─(10)─<left-part> <right-part>
├─(11)─<unsigned-integer>
└─(17)─1
└─(12)─,<unsigned-integer>
└─(17)─2
└─(7)─<constant-declaration> <constant-declarations-list>
├─(8)─<constant-identifier> = <constant>;
├─(13)─<identifier>
└─(15)─B
└─(9)─'<complex-number>'
└─(10)─<left-part> <right-part>
├─(11)─<unsigned-integer>
└─(17)─3
└─(12)─<empty>
└─(7)─<constant-declaration> <constant-declarations-list>
├─(8)─<constant-identifier> = <constant>;
└─(13)─<identifier>
└─(7)─<empty>
└─(6)─<empty>
└─(4)─<empty>

Syntax Error Missing Semicolon

examples\syntax-error-missing-semicolon.signal

```
1 PROGRAM CALC;
2 CONST
3   X = '42' (* Missing ';' after constant *)
4 BEGIN
5 END.
```

Output

```
Parser | Error [4:1]: Expected ';' at the end of constant declaration

Tokens
┌──────┬─────┬─────┬─────────────┬──────────────────────────────────────────┐
│ Code │ Row │ Col │ Type        │ Value                                    │
├──────┼─────┼─────┼─────────────┼──────────────────────────────────────────┤
│  300 │   1 │   1 │ Keyword     │ PROGRAM                                  │
│ 1000 │   1 │   9 │ Identifier  │ CALC                                     │
│   59 │   1 │  13 │ Delimiter   │ ;                                        │
│  301 │   2 │   1 │ Keyword     │ CONST                                    │
│ 1001 │   3 │   3 │ Identifier  │ X                                        │
│   61 │   3 │   5 │ Delimiter   │ =                                        │
│   39 │   3 │   7 │ Delimiter   │ '                                        │
│  500 │   3 │   8 │ Literal     │ 42                                       │
│   39 │   3 │  10 │ Delimiter   │ '                                        │
│  302 │   4 │   1 │ Keyword     │ BEGIN                                    │
│  303 │   5 │   1 │ Keyword     │ END                                      │
│   46 │   5 │   4 │ Delimiter   │ .                                        │
└──────┴─────┴─────┴─────────────┴──────────────────────────────────────────┘

Identifiers
┌──────┬──────────────────────────────────┐
│ Code │ Value                            │
├──────┼──────────────────────────────────┤
│ 1000 │ CALC                             │
│ 1001 │ X                                │
└──────┴──────────────────────────────────┘

Literals
┌──────┬──────────────────────────────────┐
│ Code │ Value                            │
├──────┼──────────────────────────────────┤
│  500 │ 42                               │
└──────┴──────────────────────────────────┘

Syntax Tree
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─CALC
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     ├─(13)─<identifier>
                  │     │           │     │      └─(15)─X
                  │     │           │     └─(9)─'<complex-number>'
                  │     │           │           └─(10)─<left-part> <right-part>
                  │     │           │                 ├─(11)─<unsigned-integer>
                  │     │           │                 │      └─(17)─42
                  │     │           │                 └─(12)─<empty>
                  │     │           └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Syntax Error Missing Symbols

examples\syntax-error-missing-symbols.signal

```
1 PROGRAM; (* Missing procedure identifier *)
2 CONST
3   X '42'; (* Missing '=' *)
4   Y = '24; (* Missing closing ''' (single quote) *)
5   Z = '54$EXP(32)' (* Missing ';' after constant *)
6   = '23,45'; (* Missing constant identifier *)
7   Q = '999';
8 BEGIN
9 . (* Missing END keyword *)
```

Output

```
Parser | Error [1:8]: Expected procedure identifier after 'PROGRAM' keyword
Parser | Error [3:5]: Expected '=' in constant declaration
Parser | Error [4:10]: Expected ''' (single quote) at the end of constant declaration
Parser | Error [6:3]: Expected ';' at the end of constant declaration
Parser | Error [6:3]: Expected constant identifier in constant declaration
Parser | Error [9:1]: Expected 'END' keyword at the end of the block
```

Tokens

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
59	1	8	Delimiter	;
301	2	1	Keyword	CONST
1000	3	3	Identifier	X
39	3	5	Delimiter	'
500	3	6	Literal	42
39	3	8	Delimiter	'
59	3	9	Delimiter	;
1001	4	3	Identifier	Y
61	4	5	Delimiter	=
39	4	7	Delimiter	'
501	4	8	Literal	24
59	4	10	Delimiter	;
1002	5	3	Identifier	Z
61	5	5	Delimiter	=
39	5	7	Delimiter	'
502	5	8	Literal	54
256	5	10	Multi-Delimiter	$EXP
40	5	14	Delimiter	(
503	5	15	Literal	32
41	5	17	Delimiter	)
39	5	18	Delimiter	'
61	6	3	Delimiter	=
39	6	5	Delimiter	'
504	6	6	Literal	23
44	6	8	Delimiter	,
505	6	9	Literal	45
39	6	11	Delimiter	'
59	6	12	Delimiter	;
1003	7	3	Identifier	Q
61	7	5	Delimiter	=
39	7	7	Delimiter	'
506	7	8	Literal	999
39	7	11	Delimiter	'
59	7	12	Delimiter	;
302	8	1	Keyword	BEGIN
46	9	1	Delimiter	.

Identifiers

Code	Value
1000	X
1001	Y
1002	Z
1003	Q

Literals

Code	Value

500	42
501	24
502	54
503	32
504	23
505	45
506	999

Syntax Tree

```
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     └─(13)─<identifier>
                  │     │           │           └─(15)─X
                  │     │           └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                 ├─(8)─<constant-identifier> = <constant>;
                  │     │                 │     ├─(13)─<identifier>
                  │     │                 │     │      └─(15)─Y
                  │     │                 │     └─(9)─'<complex-number>'
                  │     │                 │           └─(10)─<left-part> <right-part>
                  │     │                 │                 ├─(11)─<unsigned-integer>
                  │     │                 │                 │      └─(17)─24
                  │     │                 │                 └─(12)─<empty>
                  │     │                 └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                       ├─(8)─<constant-identifier> = <constant>;
                  │     │                       │     ├─(13)─<identifier>
                  │     │                       │     │      └─(15)─Z
                  │     │                       │     └─(9)─'<complex-number>'
                  │     │                       │           └─(10)─<left-part> <right-part>
                  │     │                       │                 ├─(11)─<unsigned-integer>
                  │     │                       │                 │      └─(17)─54
                  │     │                       │                 └─(12)─$EXP( <unsigned-integer> )
                  │     │                       │                        └─(17)─32
                  │     │                       └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                             ├─(8)─<constant-identifier> = <constant>;
                  │     │                             │     └─(13)─<identifier>
                  │     │                             └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │                                   ├─(8)─<constant-identifier> = <constant>;
                  │     │                                   │     ├─(13)─<identifier>
                  │     │                                   │     │      └─(15)─Q
                  │     │                                   │     └─(9)─'<complex-number>'
                  │     │                                   │           └─(10)─<left-part> <right-part>
                  │     │                                   │                 ├─(11)─<unsigned-integer>
                  │     │                                   │                 │      └─(17)─999
                  │     │                                   │                 └─(12)─<empty>
                  │     │                                   └─(7)─<empty>
                  │     └─(6)─<empty>
                  └─(4)─<empty>
```

Syntax Error Mixed

examples\syntax-error-mixed.signal

```
1 PROGRAM MIXED;
2 CONST
3   X = '10@20'; (* '@' is an invalid character - tokenizer error *)
4   Y = '5$EXP(3)'; (* Valid constant - should parse fine *)
5 BEGIN
6 END.
```

Output

```
Tokenizer | Error [3:10]: Invalid character '@'
Parser | Error [3:11]: Expected ''' (single quote) at the end of constant declaration

Tokens
```

Code	Row	Col	Type	Value
300	1	1	Keyword	PROGRAM
1000	1	9	Identifier	MIXED
59	1	14	Delimiter	;
301	2	1	Keyword	CONST
1001	3	3	Identifier	X
61	3	5	Delimiter	=
39	3	7	Delimiter	'
500	3	8	Literal	10
501	3	11	Literal	20
39	3	13	Delimiter	'
59	3	14	Delimiter	;
1002	4	3	Identifier	Y
61	4	5	Delimiter	=
39	4	7	Delimiter	'
502	4	8	Literal	5
256	4	9	Multi-Delimiter	$EXP
40	4	13	Delimiter	(
503	4	14	Literal	3
41	4	15	Delimiter	)
39	4	16	Delimiter	'
59	4	17	Delimiter	;
302	5	1	Keyword	BEGIN
303	6	1	Keyword	END
46	6	4	Delimiter	.

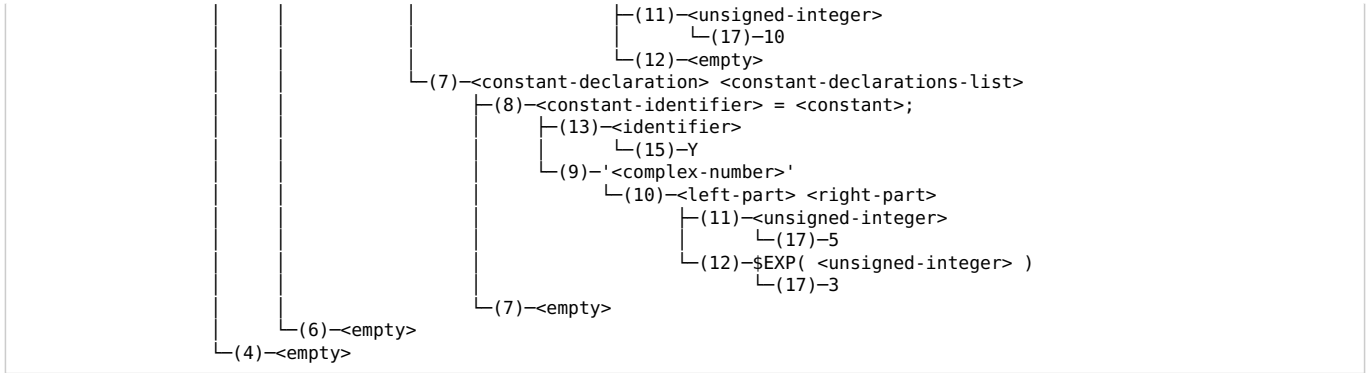
```
Identifiers
```

Code	Value
1000	MIXED
1001	X
1002	Y

```
Literals
```

Code	Value
500	10
501	20
502	5
503	3

```
Syntax Tree
<signal-program>
└─(1)─<program>
      └─(2)─PROGRAM <procedure-identifier>; <block>.
            ├─(14)─<identifier>
            │      └─(15)─MIXED
            └─(3)─<declarations> BEGIN <statements-list> END
                  ├─(5)─<constant-declarations>
                  │     ├─(6)─CONST <constant-declarations-list>
                  │     │     └─(7)─<constant-declaration> <constant-declarations-list>
                  │     │           ├─(8)─<constant-identifier> = <constant>;
                  │     │           │     ├─(13)─<identifier>
                  │     │           │     │      └─(15)─X
                  │     │           │     └─(9)─'<complex-number>'
                  │     │           │           └─(10)─<left-part> <right-part>
```


├─(11)─<unsigned-integer>
└─(17)─10
└─(12)─<empty>
└─(7)─<constant-declaration> <constant-declarations-list>
├─(8)─<constant-identifier> = <constant>;
├─(13)─<identifier>
└─(15)─Y
└─(9)─'<complex-number>'
└─(10)─<left-part> <right-part>
├─(11)─<unsigned-integer>
└─(17)─5
└─(12)─$EXP(<unsigned-integer>)
└─(17)─3
└─(7)─<empty>
└─(6)─<empty>
└─(4)─<empty>